

```
"""
```

```
SemanticFriction – Engine3-faithful dz/dt accumulator, Silver Ratio grounded.
```

```
ARCHITECTURE:
```

```
    E          = (z · z) / 2          quadratic integral (Engine3's semantic_inte
    dz_dt      = (E - E_prev) / dt    rate of change (Engine3's dz_dt)
    step_contribution = step_weight / (1 + |dz_dt| / dz_dt_floor)
    friction -= step_contribution     walks DOWN by a decimal each pass
    friction = clip(friction, 0, 1)
```

```
CLOSURE (Ω) when ALL three hold:
```

```
    friction    < closure_friction_threshold    (integral resolved)
    |dz_dt|     < dz_dt_floor                    (field stabilized at Silver floor
    |z - z*|    < closure_state_tol              (mean reached architectural inten
```

```
THRESHOLDS derived from Silver Ratio (see constants.py):
```

```
    E_target    = kT * (3 - 2√2) = kT / δ_S²
    dz_dt_floor = √(2 · kT · (3-2√2) · λ · dt) / N
```

```
WHERE 0.17 CAME FROM:
```

```
    The FourierProjection maps  $\psi(x,y) \rightarrow z = \text{mean}(\psi)$  (DC Fourier mode,  $/N^2$ ).
    So  $E = |z|^2/2 = |\text{mean}(\psi)|^2/2$  – NOT total field energy.
    At thermal equilibrium:  $E/kT = (3 - 2\sqrt{2}) = 1/\delta_S^2$  exactly.
    Observed 0.17130 vs 0.17157 – gap of 0.000273 = Monte Carlo noise.
```

```
"""
```

```
from __future__ import annotations
```

```
import math
```

```
import numpy as np
```

```
from typing import List, Optional
```

```
from dataclasses import dataclass, field
```

```
from uma.core.state import FieldPosterior
```

```
from uma.semantic.constants import (
```

```
    silver_E_target, silver_dz_dt_floor, silver_field_scale, INV_DELTA_S_SQ
```

```
)
```

```
# -----
```

```
# Config
```

```
# -----
```

```
@dataclass
```

```
class SemanticFrictionConfig:
```

```

# Step weight: how much friction decrements per pass (decimal per step)
step_weight: float = 0.02

# Closure thresholds
closure_friction_threshold: float = 0.15
convergence_dz_dt: float = None      # if None: derived from Silver Ratio
closure_state_tol: float = None      # if None: derived from field_scale * 2

# Minimum steps before closure can be declared
min_steps_before_closure: int = 8

# Physics params (set automatically from UMA Config if not supplied)
kT: float = 0.005
lam: float = 0.4
N: int = 32
dt: float = 0.04

def __post_init__(self):
    if self.convergence_dz_dt is None:
        self.convergence_dz_dt = silver_dz_dt_floor(self.kT, self.lam, self.N)
    if self.closure_state_tol is None:
        self.closure_state_tol = silver_field_scale(self.kT, self.N, 1.0) * 2

@classmethod
def from_uma_config(cls, cfg, dt: float = 0.04, **kwargs) -> "SemanticFrictio
    """Build from a UMA Config object – all thresholds Silver-derived."""
    return cls(
        kT=cfg.generic.kT,
        lam=cfg.generic.lam,
        N=cfg.grid.N,
        dt=dt,
        **kwargs,
    )

# -----
# FrictionRecord
# -----

@dataclass
class FrictionRecord:
    t: float
    E: float          #  $\langle z \cdot z \rangle / 2$  – quadratic integral
    dz_dt: float      #  $(E - E_{\text{prev}}) / dt$ 
    friction: float    # accumulated,  $\in [0, 1]$ , walks down each step
    z_distance: float  #  $|z - z_{\text{target}}| / \text{field\_scale}$ 
    closed: bool

```

```

# -----
# SemanticFriction
# -----

class SemanticFriction:
    """
    Accumulates the semantic friction integral via Engine3's dz/dt pattern.

    Friction walks from 1.0 down by a decimal at each checkpoint pass.
    The decrement per step is proportional to how much dz/dt has resolved
    relative to the Silver Ratio thermal floor.
    """

    def __init__(
        self,
        z_target: np.ndarray,
        config: Optional[SemanticFrictionConfig] = None,
    ):
        self.z_target = np.asarray(z_target, dtype=float)
        self.config = config or SemanticFrictionConfig()

        self._friction: float = 1.0
        self._E_prev: Optional[float] = None
        self._records: List[FrictionRecord] = []
        self._times: List[float] = []
        self._frictions: List[float] = []

    def reset(self, initial_posterior: FieldPosterior) -> None:
        self._friction = 1.0
        self._E_prev = None
        self._records.clear()
        self._times.clear()
        self._frictions.clear()

    def update(self, posterior: FieldPosterior, t: float) -> FrictionRecord:
        """
        One friction step - Engine3's loop translated to field coordinates.

        E          = |z|^2/2                                (semantic_integral)
        dz_dt      = (E - E_prev) / dt                      (Engine3's dz_dt)
        step       = step_weight / (1 + |dz_dt|/floor)      (decimal per pass)
        friction -= step                                    (walks down)
        """
        cfg = self.config
        z = posterior.mean

```

```

E = float(np.real(z @ z) / 2.0)

if self._E_prev is None:
    dz_dt = 0.0
else:
    dz_dt = (E - self._E_prev) / cfg.dt

self._E_prev = E

# Decrement: full step_weight when dz_dt=0, half at dz_dt=floor
floor = cfg.convergence_dz_dt + 1e-15
step = cfg.step_weight / (1.0 + abs(dz_dt) / floor)
self._friction = max(0.0, self._friction - step)

# State distance normalized to Silver field scale
field_scale = silver_field_scale(cfg.kT, cfg.N, 1.0) + 1e-15
z_dist = float(np.linalg.norm(z - self.z_target)) / field_scale

# Closure: all three criteria
closed = False
if len(self._records) >= cfg.min_steps_before_closure:
    closed = (
        self._friction < cfg.closure_friction_threshold and
        abs(dz_dt) < cfg.convergence_dz_dt and
        z_dist < cfg.closure_state_tol / field_scale
    )

rec = FrictionRecord(t=t, E=E, dz_dt=dz_dt, friction=self._friction,
                    z_distance=z_dist, closed=closed)
self._records.append(rec)
self._times.append(t)
self._frictions.append(self._friction)
return rec

@property
def value(self) -> float:
    return self._friction

@property
def is_closed(self) -> bool:
    return bool(self._records and self._records[-1].closed)

@property
def closure_step(self) -> Optional[int]:
    for i, r in enumerate(self._records):
        if r.closed:
            return i

```

```

        return None

@property
def records(self) -> List[FrictionRecord]:
    return list(self._records)

def summary(self) -> dict:
    if not self._records:
        return {"status": "no_data"}
    final = self._records[-1]
    cfg = self.config
    return {
        "sf_total": round(self._friction, 6),
        "friction_final": round(final.friction, 6),
        "E_final": round(final.E, 8),
        "E_target_silver": round(silver_E_target(cfg.kT), 8),
        "dz_dt_final": round(final.dz_dt, 8),
        "dz_dt_floor_silver": round(cfg.convergence_dz_dt, 8),
        "z_distance_final": round(final.z_distance, 4),
        "n_steps": len(self._records),
        "is_closed": self.is_closed,
        "closure_step": self.closure_step,
        "semantic_loss_pct": round(self._friction * 100, 2),
        "INV_DELTA_S_SQ": round(INV_DELTA_S_SQ, 6),
    }

```